

```
import React, { useCallback, useEffect,
useState } from 'react';
import { View, StyleSheet,
ActivityIndicator, Text } from 'react-
native';
import MapView, { UrlTile, PROVIDER_DEFAULT
} from 'react-native-maps';

import CameraMarker from
'../components/CameraMarker';
import AlertBanner from
'../components/AlertBanner';
import { useLocation } from
'../hooks/useLocation';
import { useProximityAlerts } from
'../hooks/useProximityAlerts';
import { getCachedCameraNodes,
syncCameraNodesForBounds } from
'../services/CameraDataService';
import { regionToBounds } from
'../utils/geo';
import { OSM_TILE_URL_TEMPLATE } from
'../utils/constants';

const INITIAL_REGION = {
  latitude: 33.7490, // Atlanta, as a sane
default
  longitude: -84.3880,
  latitudeDelta: 0.05,
  longitudeDelta: 0.05,
```

```

};

export default function MapScreen() {
  const { location, errorMsg: locationError
} = useLocation();
  const [cameraNodes, setCameraNodes] =
useState([]);
  const [syncError, setSyncError] =
useState(null);
  const { nodesInRange } =
useProximityAlerts(location, cameraNodes);

  const loadForRegion = useCallback(async
(region) => {
    const bounds = regionToBounds(region);

    // Offline-first: show whatever's
cached immediately.
    const cached = await
getCachedCameraNodes(bounds);
    setCameraNodes(cached);

    // Then attempt a background refresh;
failures are non-fatal since we
    // already have cached data on screen.
    try {
      await
syncCameraNodesForBounds(bounds);
      const refreshed = await
getCachedCameraNodes(bounds);
      setCameraNodes(refreshed);
      setSyncError(null);
    } catch (err) {
      setSyncError('Offline – showing

```

```

cached nodes only');
    }
    }, []);

    useEffect(() => {
        loadForRegion(INITIAL_REGION);
    }, [loadForRegion]);

    return (
        <View style={styles.container}>
            <MapView
                provider={PROVIDER_DEFAULT}
                style={styles.map}
                initialRegion={INITIAL_REGION}
                onRegionChangeComplete=
{loadForRegion}
                showsUserLocation
                showsMyLocationButton
            >
                <UrlTile urlTemplate=
{OSM_TILE_URL_TEMPLATE} maximumZ={19}
                flipY={false} />
                {cameraNodes.map((node) => (
                    <CameraMarker key={node.osmId}
node={node} />
                ))}
            </MapView>

            <AlertBanner nodesInRange=
{nodesInRange} />

            {(locationError || syncError) && (
                <View style={styles.statusBar}>
                    <Text style={styles.statusText}>

```

```

{locationError ?? syncError}</Text>
    </View>
  })

  {!location && !locationError && (
    <View style=
{styles.loadingOverlay}>
      <ActivityIndicator
color="#ff6b35" size="large" />
    </View>
  )}
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor:
'#1d1d1d' },
  map: { flex: 1 },
  statusBar: {
    position: 'absolute',
    bottom: 30,
    left: 16,
    right: 16,
    backgroundColor: '#1d1d1dcc',
    borderRadius: 8,
    padding: 10,
  },
  statusText: { color: '#8a8a8a',
textAlign: 'center' },
  loadingOverlay: {
    ...StyleSheet.absoluteFillObject,
    justifyContent: 'center',
    alignItems: 'center',
  },
});

```

```
        backgroundColor: '#1d1d1d88',  
    },  
});
```